

Causal Discovery and Machine Learning – from Assumptions to Applications

In the previous chapter, we reviewed three classes of sources of causal knowledge and discussed their main advantages and disadvantages. In this chapter, we'll focus on the last source of knowledge mentioned in the previous chapter – **causal discovery**.

We'll start by reviewing the popular assumptions behind causal discovery. Next, we'll present four broad families of methods for causal discovery and we'll introduce **gCastle** – the main Python package that we'll use in this chapter. We'll follow with a comparison of selected methods and a practical guide on how to combine causal discovery algorithms with expert knowledge.

By the end of this chapter, you will know a broad range of causal discovery methods. You'll be able to implement them using Python and gCastle and you'll understand the mechanics and implications of combining selected methods with pre-existing domain knowledge.

In this chapter, we will cover the following:

- Assumptions for causal discovery – a refresher
- Introduction to gCastle
- Constraint-based causal discovery
- Score-based causal discovery
- Functional causal discovery
- Gradient-based causal discovery
- Encoding expert knowledge

Causal discovery – assumptions refresher

The first time we mentioned causal discovery in this book was in *Chapter 1*. In *Chapter 5*, we went a little bit deeper and discussed two assumptions that are often used for causal discovery methods: **faithfulness** and **minimality**.

In this section, we'll review these assumptions and discuss other, more general assumptions that will be useful in our causal discovery journey.

Let's start!

Gearing up

Causal discovery aims at discovering (or learning) the true causal graph from observational (and sometimes interventional or mixed) data.

In general, this task is difficult but possible under certain conditions. Many causal discovery methods will require that we meet a set of assumptions in order to use them properly.

The first general assumption is one of **causal sufficiency** (or lack of **hidden confounding**). A vast majority of causal discovery methods rely on this assumption (although not all).

Another popular assumption for causal discovery is faithfulness.

Always trying to be faithful...

The **faithfulness** assumption states that if two variables are *conditionally independent* in their distributions given a third variable, they will also be *conditionally independent* in the graph that represents the data-generating process. This is represented more formally here:

$$X \perp\!\!\!\perp_p Y|Z \Rightarrow X \perp\!\!\!\perp_G Y|Z$$

This assumption is the reverse of the **global Markov property** that we use for **causal inference** and that we discussed back in *Chapter 5*. The formula says that if X and Y are independent in their distribution given Z , they will also be independent in the graph given Z .

...but it's difficult sometimes

The faithfulness assumption might be difficult to fulfill sometimes. One reason for this is the sampling error when testing for conditional independence in the finite sample size regime (Uhler et al., 2013).

Moreover, any situation where one variable influences another through two different paths and these paths *cancel out completely* will lead to the violation of faithfulness (see *Chapter 5* for more details and see Neal Brady's video for a quick reference: <https://bit.ly/BradyFaithfulness>).

That said, the probability of encountering the latter in the real world is extremely small (Sprites et al., 2000, pp. 68-69), even if it feels easy to come up with theoretical examples (Peters et al., 2017, pp. 107-108).

Minimalism is a virtue

There might exist more than one graph or **structural causal model (SCM)** that entails the same distribution.

That's a challenge for recovering the causal structure as the mapping between the structure and the distribution becomes ambiguous.

The **causal minimality** assumption is designed to address this. The assumption states that the **directed acyclic graph (DAG)** G is minimal with respect to distribution P if, and only if, G induces P , but no proper subgraph of G induces P . In other words, if graph G induces P , removing *any* edge from G should result in a distribution that is different than P .

Although not all causal discovery methods require all three assumptions that we discussed in this section, the three are likely the most frequent over a broad set of methods. We'll talk more about this when discussing particular methods in detail.

In this section, we refreshed three popular assumptions that many causal discovery methods rely on: *sufficiency*, *faithfulness*, and *minimality*.

In the next section, we'll discuss four streams of ideas that led to the development of classic and contemporary causal discovery methods.

The four (and a half) families

In this section, we'll present an overview of the four families of causal discovery methods. By the end of this section, you should have a good grasp of the four families and their core properties.

We use the word *families* rather than a more formal term such as *type*, as the distinction between the four families we'll use might be slightly vague. We follow the categorization proposed by Glymour et al. (2019) and extend it slightly to include more recent causal discovery methods not mentioned in Glymour and colleagues' paper.

The four streams

The origins of modern causal discovery can be traced back to the works of Judea Pearl and the 1987 paper he co-authored with George Rebane. The paper described a method that recovers causal structure from statistical data, assuming that the data-generating process has a poly-tree structure (Rebane and Pearl, 1987).

The ideas that followed from this research led to two parallel research streams focused around three academic centers – the **University of California, Los Angeles (UCLA)**, **Carnegie Mellon University (CMU)**, and **Stanford** (Pearl, 2009):

- **Constraint-based methods:** Researchers from UCLA and CMU focused on the approach to causal discovery based on graph independencies (understood as we described them in *Chapter 5*). We refer to such methods as constraint-based methods. This research stream led to the development of algorithms such as **inductive causation (IC)** and its implementation in a famous software package called TETRAD.
- **Score-based method:** The work at Stanford focused on the Bayesian network approach. Early research (for example, Heckerman et al., 1995) paved the way for the development of the **greedy equivalence search (GES)** algorithm (Chickering, 2020) – one of the classic examples of a score-based method.
- **Functional causal discovery:** The third line of thought was inspired by **independent component analysis (ICA)** (Hyvärinen et al., 2001) and developed by Shohei Shimizu and colleagues. We refer to this family of algorithms as functional causal discovery. Algorithms in this family leverage various aspects of functional forms of the relationships between variables in order to determine the causal direction. A classic example representing this family is the **Linear Non-Gaussian Acyclic Model (LiNGAM)** algorithm (Shimizu et al., 2006).
- **Gradient-based methods:** The fourth line of thought originates from the research focused on the idea of treating the graph space search as a continuous optimization problem. We'll refer to this fourth family as gradient-based methods, as we typically use gradient descent for optimization purposes. A classic method within this family is the **NOTEARS** algorithm (Zheng et al., 2018).

This section was titled *The four (and a half) families*. What about the remaining half?

The remaining half includes hybrid methods, reinforcement-learning-based methods, and other methods that do not fit the main four categories. This is not to say that these methods are less important.

Each of the families comes with its own benefits and challenges, yet the demarcation lines between them might sometimes be blurry.

In this section, we introduced the four (and a half) families of causal discovery algorithms – constraint-based, score-based, functional, gradient-based, and other – and provided a brief historical account of the main four categories.

We'll discuss each family in greater detail, but before we start, let's introduce the main library that we'll use in this chapter – **gCastle**.

Introduction to gCastle

In this section, we'll introduce gCastle (Zhang et al., 2021) – a causal discovery library that we'll use in this chapter. We'll introduce four main modules: models, synthetic data generators, visualization tools, and model evaluation tools. By the end of this section, you'll be able to generate a synthetic dataset of chosen complexity, fit a causal discovery model, visualize the results, and evaluate the model using your synthetic data as a reference.

Hello, gCastle!

What is gCastle?

It's an open source Python causal discovery library created by Huawei's Noah's Ark Lab. The library provides us with a comprehensive selection of modern causal discovery algorithms that include classics such as the PC algorithm, as well as cutting-edge gradient- or reinforcement-learning-based methods.

The repository (<https://bit.ly/gCastleRepo>) includes example notebooks, a list of currently available models, and basic documentation.

A great strength of gCastle is that it provides us with a unified, very intuitive, and elegant API for interacting with various causal discovery models. This makes the experimentation and model comparison seamless.

The library has four main functional components:

- Synthetic data generators
- Models
- Visualization tools
- Model evaluation tools

Let's examine them!

Synthetic data in gCastle

Synthetic data is often a natural choice for benchmarking causal discovery algorithms, as not many real-world causal datasets are publicly available. gCastle provides us with the `datasets` module, which allows us to generate synthetic data easily.

gCastle's datasets module

The two main objects in the module are `DAG` and `IIDSimulation`. The former allows us to generate graphs (structure) in the form of an **adjacency matrix**. The latter generates the actual data given an adjacency matrix and a set of parameters that define the properties of **structural equations** that determine the relationships between variables.

The `DAG` object allows us to generate five different types of graphs:

- Erdős–Rényi
- Scale-free (Barabási–Albert)
- Bipartite
- Hierarchical
- Low-rank

Graph types, graph theory, and network science

The idea of distinguishing between different types of graphs comes from graph theory and was later extended by what we know today as **network science**. The latter started with the works of two Hungarian mathematicians, Paul Erdős and Alfréd Rényi, who studied so-called **random graphs** (Erdős and Rényi, 1959). More recently, network science regained popularity with the works of Albert-László Barabási and Réka Albert (for example, Barabási and Albert, 1999; Barabási, 2009).

The `IIDSimulation` object allows us to choose between linear and non-linear assignments. Linear datasets can be binary (`logistic`) or continuous. Continuous datasets can be created using one of four different noise distributions:

- Gaussian (`gauss`)
- Exponential (`exp`)
- Gumbel (`gumbel`)
- Uniform (`uniform`)

Non-linear datasets can be generated using the following:

- **Multilayer perceptron** (`mip`)
- Two versions of **Gaussian processes** (`gp` and `gp-add`)
- Quadratic function (`quadratic`)
- Multiple index model (`mim`; Zheng et al., 2020)

There's also an additional simulator class called `THPSimulation`, which allows for simulating event sequences with **Topological Hawkes Processes (THP)**. The discussion on THP is beyond the scope of this book. To learn more, check Cai et al. (2021).

Let's generate some data. On the way, we'll also briefly discuss scale-free graphs and the main parameters of the `IIDSimulation` object.

Generating the data with gCastle

Let's start with the imports. The code for this chapter can be found in the notebook at <https://bit.ly/causal-ntbk-13>.

We import the `DAG` and `IIDSimulation` objects from `gCastle`. Additionally, we import `NetworkX` and `Matplotlib` for visualizations:

```
from castle.datasets import DAG, IIDSimulation
import networkx as nx
import matplotlib.pyplot as plt
```

We set the seed to keep the code reproducible:

```
SEED = 18
np.random.seed(SEED)
```

Now, we're ready to generate our first adjacency matrix:

```
adj_matrix = DAG.scale_free(
    n_nodes=10,
    n_edges=17,
    seed=SEED
)
```

We use the `.scale_free()` method of the `DAG` object. We specify the number of nodes to be 10 and the number of edges to be 17. We fix the random seed to the predefined value.

Scale-free networks have a number of interesting properties. They are created using a probabilistic **preferential attachment** mechanism. A node with a large number of connections is more likely to gain new connections than a node with a smaller number of connections.

This is sometimes metaphorically described as *the rich get richer*. The distribution of node degrees (number of edges per node) in scale-free networks is fat-tailed. This is because scale-free networks tend to have a few strongly connected hubs and a majority of nodes have a significantly smaller number of connections.

Although scale-free networks were known at least from the 1960s (although not necessarily under their current name), they significantly gained popularity in the late 1990s and early 2000s with the works of Albert-László Barabási and Réka Albert, who proposed that scale-free networks can be used to model the internet interlink structure and other real-world systems (Barabási and Albert, 1999; Barabási, 2009).

Graphs, networks, and adjacency matrices

In network science terms, *graph* and *network* are used interchangeably. As we discussed in *Chapter 4*, graphs can be represented as adjacency matrices, and matrices can be visualized as graphs. We extensively use this correspondence in the current chapter.

Let's plot the graph representing our adjacency matrix and see whether we can recognize the preferential attachment scheme in the plot. We'll use NetworkX for this purpose.

First, let's transform the adjacency matrix into the NetworkX directed graph object:

```
g = nx.DiGraph(adj_matrix)
```

Next, let's plot the graph *g*:

```
plt.figure(figsize=(12, 8))
nx.draw(
    G=g,
    node_color=COLORS[0],
    node_size=1200,
    pos=nx.circular_layout(g)
)
```

We set the figure size using Matplotlib and pass a number of parameters to NetworkX's `nx.draw()` method. We pass *g* as a graph, set the node color and size, and define the network layout to increase readability. The circular layout should provide good visual clarity for our relatively small graph.

To learn about other layout options in NetworkX, check <https://bit.ly/NetworkXLayouts> or the NetworkX documentation (<https://bit.ly/NetworkXDrawingDocs>).

Figure 13.1 presents the resulting graph:

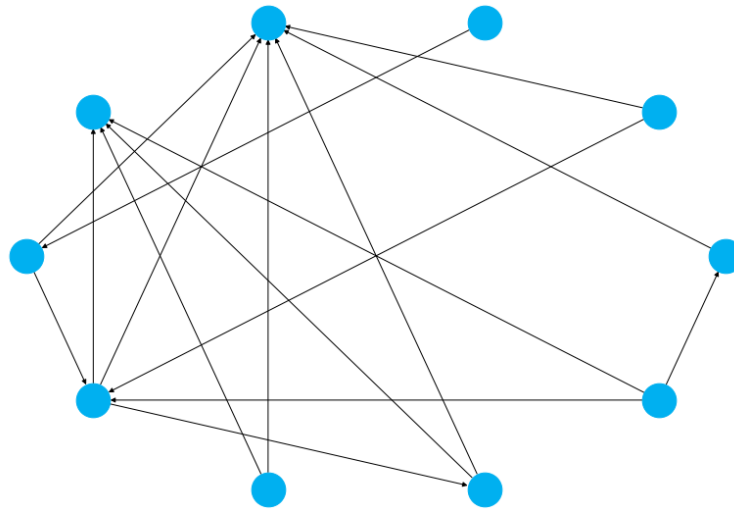


Figure 13.1 – A graph representing the adjacency matrix, g

In *Figure 13.1*, at the top, we see a highly connected node with six incoming edges. This is the effect of the preferential attachment mechanism. If we grew our graph further, the differences between highly connected nodes and less connected nodes would become even more visible. Looking at the graph in *Figure 13.1*, you can imagine how the preferential attachment mechanism would add more edges to the more connected nodes and fewer edges to the less connected ones.

We are now ready to generate some data. We'll use our adjacency matrix, `adj_matrix`, and generate 10,000 observations with linear structural assignments and Gaussian noise:

```
dataset = IIDSimulation(
    W=adj_matrix,
    n=10000,
    method='linear',
    sem_type='gauss'
)
```

The parameter `W` accepts adjacency matrices as NumPy arrays. Matrices can be (but do not have to be) weighted, `n` determines the number of samples returned, and we can choose between linear and non-linear datasets via the `method` parameter (available options are `linear` and `nonlinear`). Finally, `sem_type` determines the key characteristics of the structural assignments (`gauss`, `exp`, `mlp`, etc.).

The `dataset` object produced by `IIDSimulation` has two attributes: `X` and `B`. The former contains the actual data, and the latter contains the adjacency matrix.

We can access the generated data as follows:

```
dataset.X
```

With our dataset generated, let's learn how to fit a gCastle causal discovery model.

Fitting your first causal discovery model

gCastle offers a broad variety of causal discovery models, from classics such as PC and GES models up to recent gradient-based methods such as GOLEM.

For the sake of this demonstration, we will use the PC algorithm.

Let's start with the imports:

```
from castle.algorithms import PC
```

After importing the model class, we need to instantiate it.

```
pc = PC()
```

Training the model is extremely simple:

```
pc.learn(dataset.X)
```

The output of a causal discovery algorithm is a learned adjacency matrix. Depending on the method you use, this matrix can be either weighted or unweighted. In our case, we learned an unweighted adjacency matrix. Let's access it:

```
pc.causal_matrix
```

This will print out the learned causal matrix. We won't print it here to save space, but you can see the matrix in this chapter's notebook.

Now that we have learned the adjacency matrix, we are ready to compare it with the ground truth. First, we'll do it visually.

Visualizing the model

In the previous subsection, we visualized the adjacency matrix as a graph. Let's do the same for the true DAG and the learned DAG to compare them. *Figure 13.2* presents the result:

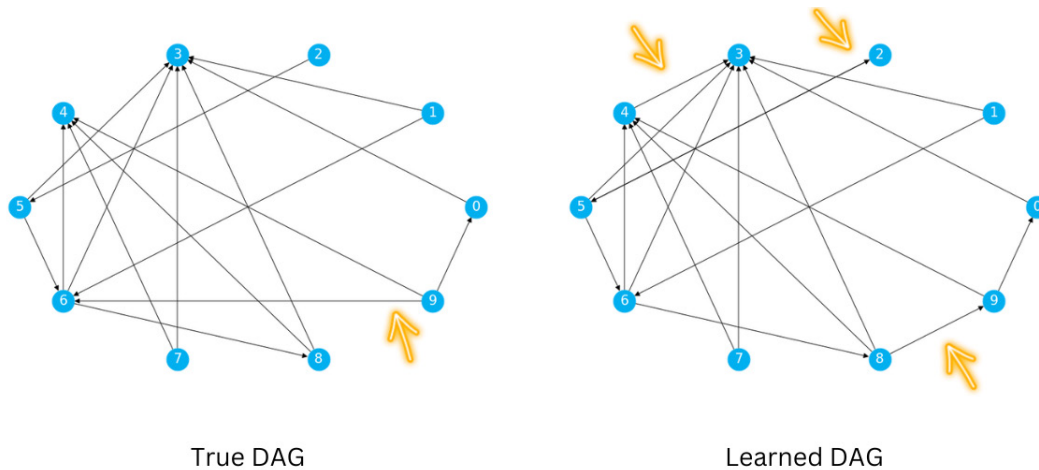


Figure 13.2 – True DAG versus learned DAG

We added yellow arrows in *Figure 13.2* to denote incorrectly identified edges. Note that the edge between nodes 2 and 5 exists in both the true and the learned DAGs, but in the learned DAG, this edge is *bidirectional*. This is a practical example of a situation where a causal discovery model returns a **complete partially directed acyclic graph (CPDAG)** (recall our discussion on **Markov equivalence classes** in *Chapter 5*).

Comparing two graphs visually is only useful for smaller graphs. Although our graph has only 10 nodes and 17 edges, the comparison might be already challenging for some of us.

Another way to visualize adjacency matrices is to use heatmaps. Because our adjacency matrix is unweighted, a respective heatmap will be binary. In a binary heatmap, each cell can only have one of two values (0 or 1).

gCastle comes with a convenience object called `GraphDAG` that plots comparison heatmaps for us. Let's import `GraphDAG` and see it in action:

```
from castle.common import GraphDAG
GraphDAG(
    est_dag=pred_dag,
    true_dag=adj_matrix)
```

You can see the resulting plot in *Figure 13.3*:

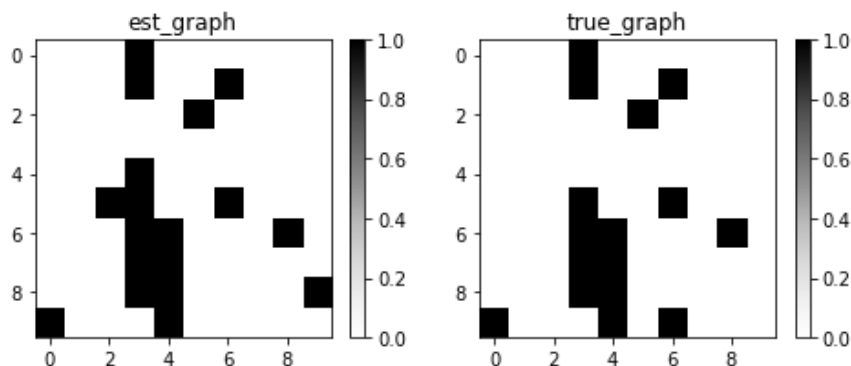


Figure 13.3 – Heatmaps representing the learned and the true adjacency matrices

After a short inspection, you should be able to spot four differences between the two matrices in *Figure 13.3*. That's congruent with our observation from *Figure 13.2*.

Although heatmaps might make it easier for us to make visual comparisons between slightly bigger graphs, they only work to a point.

Visual comparisons might be useful and even very helpful in certain cases, yet their usefulness is limited due to the limitations of our human attention, which can only track between three and seven elements at a time. That's when numerical evaluation metrics become useful.

Model evaluation metrics

gCastle comes with a dedicated object for model evaluation called `MetricsDAG`.

Let's import the object and instantiate it:

```
from castle.metrics import MetricsDAG
metrics = MetricsDAG(
    B_est=pred_dag,
    B_true=adj_matrix
)
```

The newly created `metrics` object computes a number of useful metrics for us internally. We can access them through a dictionary internally stored as an attribute called `metrics`. The values for all available metrics can be easily accessed via respective dictionary keys.

For example, to access the F1 score, we do it in the following way:

```
metrics.metrics['F1']
```

The F1 score for the PC algorithm on our graph is as follows:

```
0.8824
```

As of the time of writing (gCastle, version 1.0.3), there are nine available metrics in MetricsDAG.

We can access the complete metrics dictionary as follows:

```
metrics.metrics
```

This will give us the following:

```
{'fdr': 0.1176,
 'tpr': 0.9375,
 'fpr': 0.069,
 'shd': 3,
 'nnz': 17,
 'precision': 0.8333,
 'recall': 0.9375,
 'F1': 0.8824,
 'gscore': 0.75}
```

Table 13.1 presents a summary and definitions of the metrics available in gCastle 1.0.3:

Name	Acronym	Definition
False Discovery Rate	fdr	$\frac{rev + FP}{TP + FP}$
True Positive Rate	tpr	$\frac{TP}{TP + FN}$
False Positive Rate	fpr	$\frac{rev + FP}{TN + FP}$
Structural Hamming Distance	shd	No. of edge additions, flips, or deletions to get from the predicted graph to the true one
No. of Non-Negative Entries	nnz	$TP + FP$
Precision	precision	$\frac{TP}{TP + FP}$
Recall	recall	$\frac{TP}{TP + FN}$
F1 Score	F1	$2 \frac{recall \cdot precision}{recall + precision}$
G-Score	gscore	$\max\left(0, \frac{TP - FP}{TP + FN}\right)$

Table 13.1 – Summary of metrics available in the MetricsDAG object

In Table 13.1, *TP* stands for the number of **true positives**, *FP* stands for the number of **false positives**, *TN* stands for the number of **true negatives**, and *FN* stands for the number of **false negatives**; *rev* represents the number of **reversed edges**.

We would count an entry in the adjacency matrix as a *true positive* if we predicted that this entry should be 1 and it is also 1 in the true graph. On the other hand, we say that an entry is a *true negative* when we predicted it to be 0 and it's also 0 in the true graph. If we predicted an entry as 1 but in fact, it's 0, we count it as a *false positive*. If we predicted an entry as 0 but in fact, it's 1, we count it as a *false negative*. Reversed edges are the ones that exist in the predicted and the true graph, but we mispredicted their orientation (direction).

We'll discuss how to understand some of the metrics in the subsequent sections.

There are two other useful quantities that you might want to compute when benchmarking causal discovery methods: the number of undirected edges and the **structural intervention distance (SID)** (Peters and Bühlmann, 2015).

We can implement the former, for instance, this way:

```
def get_n_undirected(g):
    total = 0
    for i in range(g.shape[0]):
        for j in range(g.shape[0]):
            if (g[i, j] == 1) and (g[i, j] == g[j, i]):
                total += .5
    return total
```

The latter is available in the CDT Python package (<https://bit.ly/CDTMetricsDocs>) as a wrapper for an R implementation.

Now, let's summarize what we've learned in this section.

In this section, we introduced gCastle, a causal discovery library that we will use throughout this chapter. We've discussed the four main modules of the package: the synthetic data generation module, models module, data visualization module, and model evaluation module. We introduced data simulation and model fitting APIs, and demonstrated how to use the `GraphDAG` class for visualizations and the `MetricsDAG` class for calculating relevant metrics. We defined the metrics available in gCastle and introduced two additional quantities that can help us assess our model's quality: the number of undirected edges and SID.

We'll learn more about gCastle's modules in the next sections while working with specific algorithms.

Ready?

Let's start!

Constraint-based causal discovery

In this section, we'll introduce the first of the four families of causal discovery methods – constraint-based methods. We will learn the core principles behind constraint-based causal discovery and implement the **PC algorithm** (Sprites et al., 2000).

By the end of this chapter, you will have a solid understanding of how constraint-based methods work and you'll know how to implement the PC algorithm in practice using gCastle.

Constraints and independence

Constraint-based methods (also known as **independence-based methods**) aim at decoding causal structure from the data by leveraging the independence structure between three basic graphical structures: **chains**, **forks**, and **colliders**.

Let's start with a brief refresher on chains, forks, and colliders. *Figure 13.4* presents the three structures:

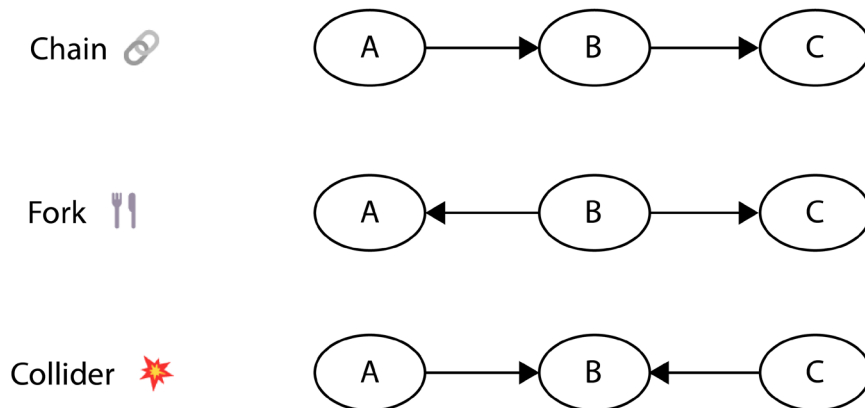


Figure 13.4 – The three basic graphical structures

In *Chapter 5*, we demonstrated that the collider structure has a unique property.

Let's take a look at *Figure 13.4* once again. Each structure consists of three variables represented by nodes A, B, and C. For each structure, let's imagine a model where we regress C on A.

When we control B in chains and forks, A and C become independent. A collider is different. When we control B in a collider, A and C become dependent.

This property can help us recover structural information from observational data.

Let's see how this can be useful in practice.

Leveraging the independence structure to recover the graph

We'll start with a graph. *Figure 13.5* presents the graph that we will use in this section. The graph topology follows the one from Glymour et al. (2019):

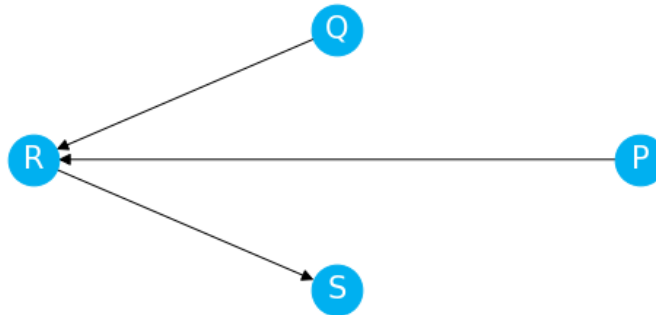


Figure 13.5 – An example graph that we'll use in this section

The graph in *Figure 13.5* consists of four nodes and three directed edges.

Let's pretend that we observe four variables that correspond to the P , Q , R , and S nodes in the graph and that we don't know the actual structure.

We'll use the **PC** algorithm to retrieve this structure from observations. The source of the algorithm's name is the first names of its creators, **Peter** (Sprites) and **Clark** (Glymour). The name likely also refers to a precursor algorithm called **IC**, proposed by Verma and Pearl in 1990.

The PC algorithm consists of five basic steps:

1. Start with a fully connected graph.
2. Delete the edges between unconditionally independent variables.
3. Iterate over all remaining variable pairs (A, B) and delete an edge between A and B if $A \perp\!\!\!\perp B | C$, where C is a conditioning set of size 1 (containing only one variable). Repeat this step for all remaining pairs (A, B) , increasing the size of the conditioning set C by 1.
4. For each triple of variables, $A - B - C$, where “ $-$ ” represents an undirected edge and where A and C are not adjacent, orient the edges $A \rightarrow B \leftarrow C$ whenever $A \not\perp\!\!\!\perp C | B$ (note that if the latter is true, we know that $A - B - C$ forms a collider and hence we know how to orient the edges).
5. For each triple of variables, $A \rightarrow B - C$, where A and C are not adjacent, orient the edge between B and C as $B \rightarrow C$. This is called **orientation propagation** (Glymour et al., 2019).

For a more detailed view of the PC algorithm, check Glymour et al. (2019), Sprites et al. (2000), or Le et al. (2015).

Let's apply the PC algorithm step-by-step to the graph presented in *Figure 13.5*.

We present the entire process in *Figure 13.6*:

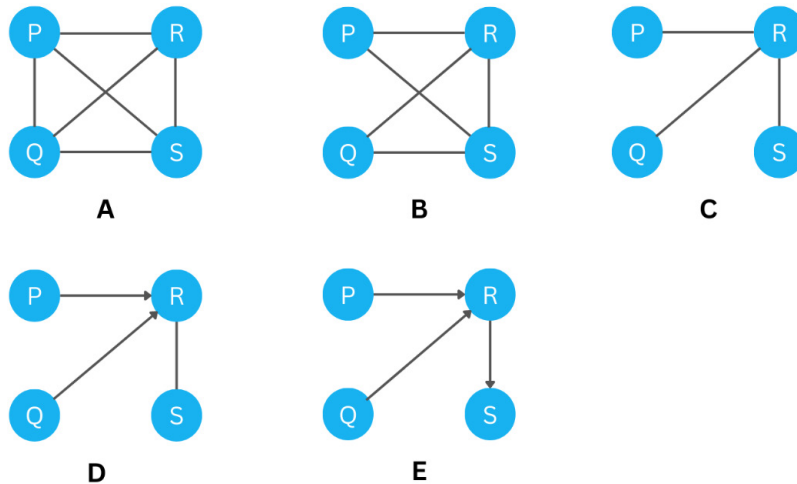


Figure 13.6 – The PC algorithm step by step

Let's examine the process in *Figure 13.6*:

1. We start with a fully connected graph (A).
2. We remove the edges between the nodes that are unconditionally independent (B). There's only one pair like this. This is the pair (P, Q).
3. We remove the edges between P and Q, and Q and S, as $P \perp\!\!\!\perp Q|R$ and $Q \perp\!\!\!\perp S|R$ (C).
4. We orient the edges $P \rightarrow R \leftarrow S$ because R is a collider between P and S (D).
5. Finally, we orient the edge between R and S because we know that R is not a collider between S and any other adjacent variable (E).

If any of the steps seem unclear to you, feel free to go through the stages in *Figure 13.6* once again and review the PC algorithm steps that we discussed earlier. If you need a deeper refresher on conditional independencies, review *Chapter 5*.

Let's generate some data according to our graph and fit the PC algorithm to recover the true structure.

We'll start by generating 1,000 observations and store them in a NumPy matrix:

```
N = 1000
p = np.random.randn(N)
q = np.random.randn(N)
r = p + q + .1 * np.random.randn(N)
s = .7 * r + .1 * np.random.randn(N)
```

```
# Store the data as a matrix
pc_dataset = np.vstack([p, q, r, s]).T
```

Now, let's instantiate and fit the algorithm:

```
pc = PC()
pc.learn(pc_dataset)
```

Let's display the predicted causal matrix alongside the true DAG:

```
GraphDAG(
    est_dag=pc.causal_matrix,
    true_dag=pc_dag)
plt.show()
```

Figure 13.7 presents the visualization:

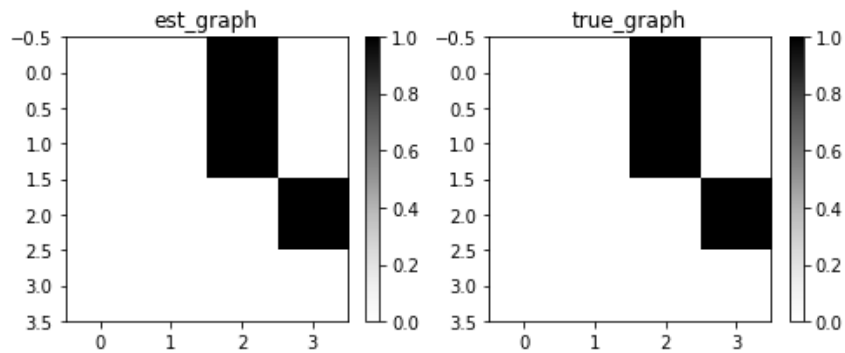


Figure 13.7 – The comparison between the DAG learned by the PC algorithm (left) and the true DAG (right)

Both DAGs in Figure 13.7 look identical. It indicates that our model learned the underlying DAG perfectly.

In this case, it's just a pure formality; nonetheless, let's print out the metrics:

```
MetricsDAG(
    B_est=pc.causal_matrix,
    B_true=pc_dag
).metrics
```

This results in the following:

```
{'fdr': 0.0,
 'tpr': 1.0,
 'fpr': 0.0,
```

```
'shd': 0,  
'nnz': 3,  
'precision': 1.0,  
'recall': 1.0,  
'F1': 1.0,  
'gscore': 1.0}
```

The metrics confirm that the model learned a perfect representation. Precision, recall, TPR, and F1 score are all equal to 1. FDR, FPR, and SHD are equal to 0. NNZ is equal to 3, indicating that there are 3 edges in the graph.

PC algorithm – hidden challenges

The PC algorithm worked very well for the data in our example. Although in general, the PC algorithm comes with theoretical guarantees of convergence to the true MEC in the large sample limit, assuming that all model assumptions are met, in certain high-dimensional cases, the *original algorithm* might return incorrect results.

The reason for this is that in certain scenarios, the algorithm might be sensitive to the order in which we perform conditional independence tests. The sensitivity has been addressed in a variant of the PC algorithm called **PC-stable** (Colombo and Maathuis, 2012).

Fortunately, implementing PC-stable is very easy with gCastle. We only need to specify a variant when initializing the object:

```
pc_stable = PC(variant='stable')
```

More on PC-stable

In fact, PC-stable only addresses a part of the problem of ordering sensitivity – the variant is only insensitive to the ordering of the tests that lead to the discovery of the skeleton but is still sensitive to ordering when finding colliders and orienting edges. Other variants of the PC algorithm have been proposed to address these two problems, but it turns out that, in practice, they might lead to less useful results than simple PC-stable. Check Colombo and Maathuis (2012) for more details.

Another challenge for the PC algorithm is computational. Performing a significant number of conditional independence tests in a very large dataset might be computationally expensive. One way to address this challenge is to parallelize the algorithm (Le et al., 2015). Fortunately, gCastle provides us with a parallelized version out of the box.

To use the parallel version, simply specify the variant:

```
pc_parallel = PC(variant='parallel')
```

The results for the stable and parallel versions of the algorithm are identical for our example so we do not present them here (but you can find them in the notebook).

PC algorithm for categorical data

In our example, we used continuous data, but the PC algorithm can also work for categorical data. Recall that the main principle of how PC works is independence testing. By default, PC uses Fisher's z-test (Fisher, 1921) in order to test independencies between variables. Fisher's z-test works well for continuous and linear-Gaussian data.

For categorical variables, we need another approach.

gCastle provides us with a number of independence tests that can be used in this context. Two of them, χ^2 and G^2 (Tsamardinos et al., 2006), can be used directly in the PC algorithm, by simply passing a string to the `ci_test` parameter.

For the χ^2 test, use the following:

```
pc_cat = PC(ci_test='chi2')
```

For the G^2 test, use the following:

```
pc_cat = PC(ci_test=g2)
```

We can find more tests in the `castle.common.independence_tests.CITest` object. The tests include Neyman's test, the Cressie-Read test (Cressie and Read, 1984), and more.

In order to use them, we need to pass a function (without calling it) to the `ci_test` parameter:

```
from castle.common.independence_tests import CITest
pc_cat_alt = PC(ci_test=CITest.cressie_read)
```

We can also use an arbitrary custom test as long as it returns a three-tuple where the last element represents the p -value of the test – structurally speaking: `(_, _, p_val)`.

This modular design is really powerful. Notice that all causal libraries used in this book so far – DoWhy, EconML, and gCastle – promote an open, modular, and flexible design, which makes them well suited for future developments and facilitates research.

The time has come to conclude this section. We'll get back to the PC algorithm briefly in later sections of this chapter to reveal one more hidden gem that can be really helpful in real-world scenarios.

In this section, we learned about constraint-based causal discovery methods and introduced the PC algorithm. We discussed the five basic steps of the PC algorithm and applied them to our example DAG. Next, we generated data according to the DAG and implemented the PC algorithm using gCastle.

We discussed two limitations of the algorithm and introduced two variants of the PC algorithm aiming to address them. Finally, we discussed how to use alternative conditional independence tests including the ones provided by the library and the custom ones.

In the next section, we'll introduce the second major family of causal discovery methods – score-based methods.

Score-based causal discovery

In this section, we'll introduce **score-based methods** for causal discovery. We'll discuss the mechanics of the **GES** algorithm and implement it using gCastle.

Tabula rasa – starting fresh

The very first step of the PC algorithm was to build a fully-connected graph. GES starts on the other end of the spectrum, but first things first.

GES is a two-stage procedure. First, it generates the edges, then it prunes the graph.

The algorithm starts with a blank slate – an entirely disconnected graph – and iteratively adds new edges. At each step, it computes a score that expresses how well a new graph models the observed distribution, and at each step, the edge that leads to the highest score is added to the graph.

When no more improvement can be achieved, the pruning phase begins. In this phase, the algorithm removes edges iteratively and checks for score improvement. The phase continues until no further improvement can be achieved by removing edges.

The entire first and second phases are executed in a greedy fashion (hence the *greedy* part of the algorithm's name).

GES might return a CPDAG, which corresponds to a Markov equivalence class for a given DAG (hence the *equivalence* part of the name).

GES – scoring

In order to compute the score at each step, Chickering (2003) uses the Bayesian scoring criterion. Huang et al. (2018) proposed a more general method based on regression in **Reproducing Kernel Hilbert Space (RHKS)**, which allows for mixed data types and multidimensional variables.

gCastle's implementation offers two scoring criteria: **Bayesian Information Criterion (BIC or bic)** and **Bayesian Dirichlet Equivalent Uniform (Bdeu or bdeu)**. The latter is intended to work with discrete variables exclusively.

GES in gCastle

Implementing the GES algorithm in gCastle is straightforward:

```
ges = GES(criterion='bic')
```

To train the model, use the `.learn()` method (analogously to what we've done for the PC algorithm):

```
ges.learn(pc_dataset)
```

We used the dataset generated according to the graph in *Figure 13.5*. Let's plot the results to see how GES performed:

```
GraphDAG(
    est_dag=ges.causal_matrix,
    true_dag=pc_dag)
plt.show()
```

The resulting plot is presented in *Figure 13.8*:

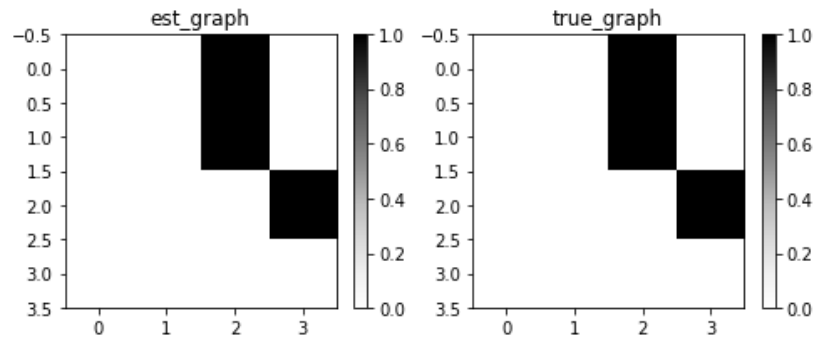


Figure 13.8 – Results of the GES algorithm

As we can see in *Figure 13.8*, the GES algorithm did a very good job and retrieved the perfect graph.

In general, the GES algorithm can be proven to be optimal (based on the so-called **Meek Conjecture**; see Chickering (2003) for details), but the guarantees are only asymptotic.

In the limit of a large sample, GES converges to the same solution as PC. In my personal experience, I found GES repeatedly underperforming compared to other methods (including PC), but please note that my observations do not have a systematic character here.

As a final remark, we need to highlight that GES requires that there's no hidden confounding in our dataset.

In this section, we introduced score-based causal discovery methods, discussed the GES algorithm, and applied it to our PC dataset. In the next section, we'll cover functional causal discovery methods.

Functional causal discovery

Functional causal discovery (also called **function-based causal discovery**) is all about leveraging the information about the *functional forms* and *properties of distributions* governing the relationships between variables in order to uniquely identify causal directions in a dataset. In this section, we'll introduce the logic behind function-based methods, using the **Additive Noise Model (ANM)** (Hoyer et al., 2008) and **LiNGAM** (Shimizu et al., 2006) as examples. We'll implement ANM and LiNGAM and discuss the differences between the two. By the end of this section, you will have a good understanding of the general principles of function-based causal discovery and you'll be able to apply the ANM and LiNGAM models to your own problems using Python and gCastle.

The blessings of asymmetry

*Tyger Tyger, burning bright,
In the forests of the night;
What immortal hand or eye,
Could frame thy fearful symmetry?*

William Blake – Tyger (1794)

(Blake, 2009)

Symmetry plays a special role in human life. We rate symmetric faces as more beautiful (Grammer and Thornhill, 1994), we use symmetry in architecture (*Figure 13.9*) and in everyday design such as cars, planes, and trains, and have a preference for symmetrical objects (Enquist and Arak, 1994).



Figure 13.9 – Three famous structures leveraging symmetry: Taj Mahal in India (top left; image by Maahid Photos), Pyramids in Egypt (bottom left; image by Diego Ferrari), Notre Dame in France (right; image by Max Avans) – all images can be found at Pexels.com

Iain Johnston and colleagues argue that symmetry is fundamentally connected to the process of evolution and its “preference” to compress information (Johnston et al., 2021).

That said, symmetry only tells a part of the story. Perfectly symmetrical faces and objects are perceived as unnatural – *fearfully symmetrical*. It seems that nature likes to be asymmetrical sometimes and a large class of real-world variables is distributed in a non-symmetrical manner.

In *Chapter 3*, we demonstrated that when we have two causally and linearly-related normally distributed variables, we have no way to decide which direction is causal, based solely on the data. The behavior of the regression in both directions is *symmetrical* in the sense that the residuals in both cases are independent, not containing any information that we could leverage to decode the causal direction. In other words, the linear-Gaussian causal mechanism does not leave any *traces* in pair-wise relationships.

Is there anything we can do about it?

It seems that the linear-Gaussian case is hard, but moving into either the non-linear or non-Gaussian realm *breaks the symmetry*.

Let’s see how.

ANM model

ANM (Hoyer et al., 2008) is a causal discovery algorithm that leverages the fact that when two variables are related non-linearly, the causal mechanism *does leave traces*.

Let’s see it in practice.

We’ll start by generating some non-linear data:

```
x = np.random.randn(1000)
y = x**3 + np.random.randn(1000)
```

Let’s plot it:

```
plt.figure(figsize=(10, 7))
plt.scatter(x, y, alpha=.5, color=COLORS[0])
plt.xlabel('$X$')
plt.ylabel('$Y$')
plt.show()
```


Figure 13.10 shows the results:

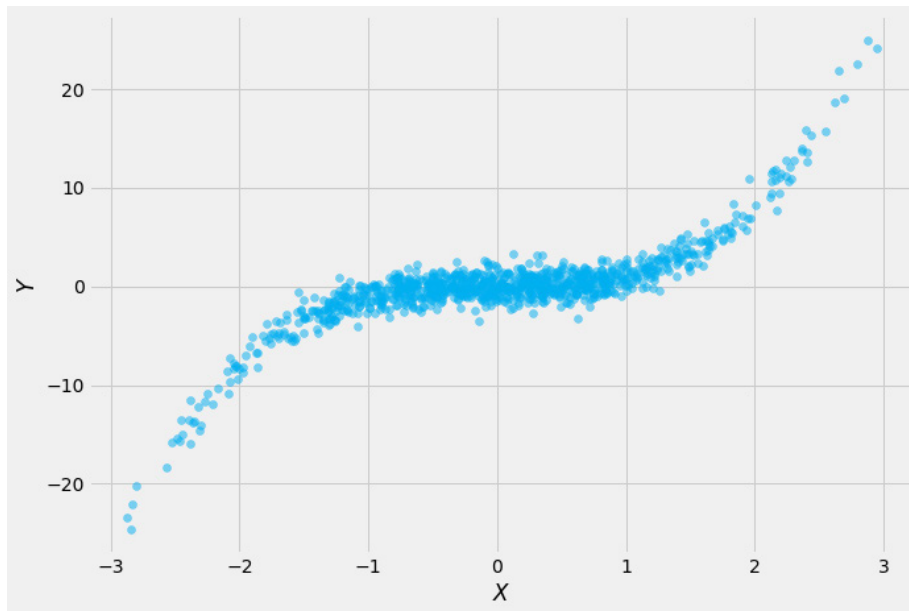


Figure 13.10 – Scatter plot of our non-linear dataset

Now, let's fit two non-linear **spline regressions** to our data – one in the causal ($X \rightarrow Y$) and one in the anti-causal ($Y \rightarrow X$) direction.

We'll use 150 splines in each model:

```
n_splines = 150

# Instantiate the models
model_xy = GAM(n_splines=n_splines)
model_yx = GAM(n_splines=n_splines)

# Fit the models
model_xy.fit(x.reshape(-1, 1), y)
model_yx.fit(y.reshape(-1, 1), x)

# Generate predictions
y_pred = model_xy.predict(x.reshape(-1, 1))
x_pred = model_yx.predict(y.reshape(-1, 1))
```

Let's plot the data alongside the fitted regression curves. *Figure 13.11* presents the results:

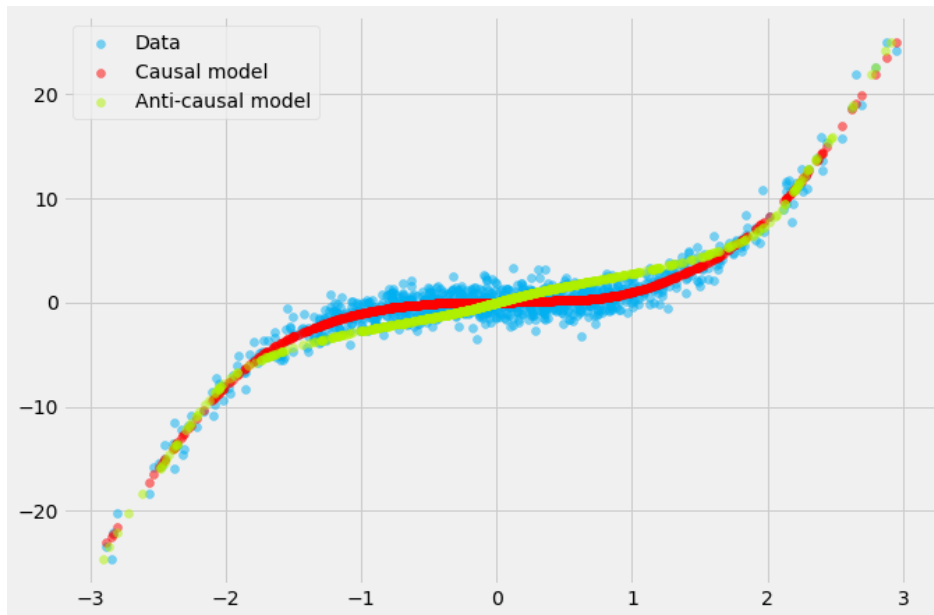


Figure 13.11 – Scatter plot of non-linear data with two fitted regression curves

The two lines representing the causal and the anti-causal models differ significantly.

Let's compute the residuals for both models:

```
residuals_xy = y - y_pred
residuals_yx = x - x_pred
```

Let's now plot them:

```
plt.figure(figsize=(15, 7))

plt.subplot(121)
plt.scatter(x, residuals_xy, alpha=.5, color=COLORS[0])
plt.xlabel('$X$', fontsize=14)
plt.ylabel('$Y$-residuals$', fontsize=14)

plt.subplot(122)
plt.scatter(residuals_yx, y, alpha=.5, color=COLORS[0])
plt.xlabel('$X$-residuals$', fontsize=14)
plt.ylabel('$Y$', fontsize=14)

plt.show()
```

Figure 13.12 presents the results:

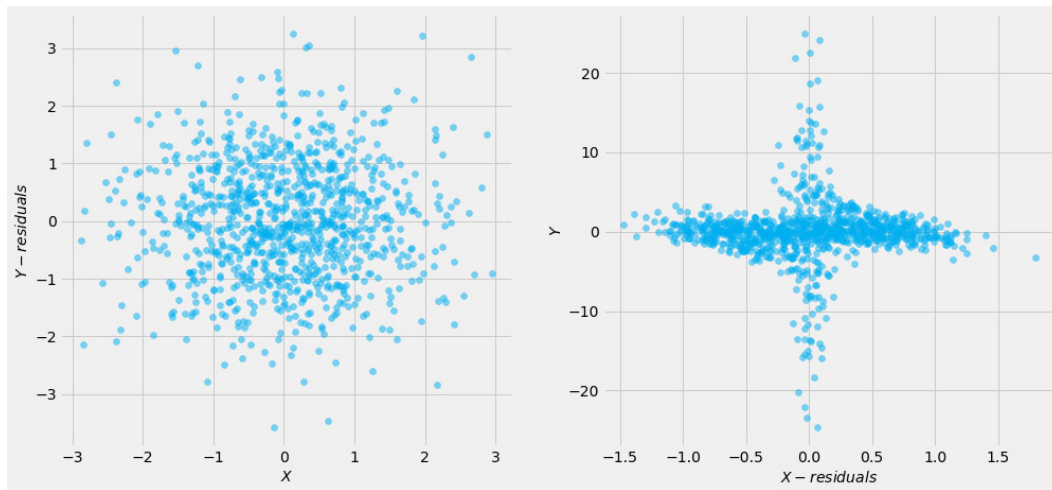


Figure 13.12 – Scatter plots of residuals for the causal and the anti-causal model

As we can see in *Figure 13.12*, the residuals for both models form very different patterns. In particular, the residuals for the anti-causal model (right panel) form a cross-like pattern, indicating a lack of independence between the residuals (x axis) and the predictor (y axis).

This is great news for us!

If we can algorithmically decide which patterns indicate independence, we can retrieve the information about the causal direction.

Assessing independence

It seems clear that none of the traditional correlation metrics such as Pearson's r or Spearman's ρ will be able to help here. The relationship in the right panel of *Figure 13.12* is non-monotonic, violating the basic assumption of both methods.

Luckily, many methods exist that can help us with this task. One of the methods traditionally used in ANM models is called **Hilbert-Schmidt Independence Criterion (HSIC)**. HSIC can easily handle patterns like the one that we obtained for the anti-causal model.

Let's compute HSIC for both models. We'll use the implementation of HSIC provided by gCastle.

Let's import the function first:

```
from castle.common.independence_tests import hsic_test
```

Now, let's compute the HSIC statistic:

```
# Compute HSIC
is_indep_xy = hsic_test(
    x = x.reshape(-1, 1),
    y = residuals_xy.reshape(-1, 1),
    alpha=.05
)

is_indep_yx = hsic_test(
    x = y.reshape(-1, 1),
    y = residuals_yx.reshape(-1, 1),
    alpha=.05
)
```

gCastle's implementation of HSIC returns 1 when two variables are independent and 0 when they are dependent, given a specified significance level α . We use the `.reshape()` method to reshape our arrays into two-dimensional arrays, as required by the API.

Let's print the results:

```
is_indep_xy, is_indep_yx
```

This gives us the following:

```
(1, 0)
```

The result says that the residuals are independent for the model in the $X \rightarrow Y$ direction, while they are dependent for the model in the $Y \rightarrow X$ direction.

We expect the residuals from the causal model to be independent. Hence, we can conclude that, according to our model, the true causal model is the one in the $X \rightarrow Y$ direction.

This is correct!

Congrats, you just learned how to implement the ANM model from scratch!

There are also other options to implement ANM. You can use another non-linear model instead of spline regression (for instance, gCastle's implementation of ANM uses Gaussian process regression). You can also use other tools to compare the residuals. For instance, an alternative approach to independence testing is based on likelihood. Check this notebook for implementation: <https://bit.ly/ANMNotebook>.

ANM only works when the data has independent additive noise; it also requires no hidden confounding.

To implement ANM using gCastle, use the `ANMNonlinear` class.

LiNGAM time

While ANM relies on non-linearity to break the symmetry between the causal and anti-causal models, LiNGAM relies on non-Gaussianity.

Let's compare residual patterns in two linear datasets: one with Gaussian and one with non-Gaussian noise.

Let's start by generating the data:

```
SAMPLE_SIZE = 1000

x_gauss = np.random.normal(0, 1, SAMPLE_SIZE)
y_gauss = x_gauss + 0.3 * np.random.normal(0, 1,
                                           SAMPLE_SIZE)

x_ngauss = np.random.uniform(0, 1, SAMPLE_SIZE)
y_ngauss = x_ngauss + 0.3 * np.random.uniform(0, 1,
                                                SAMPLE_SIZE)
```

We used the uniform distribution in the non-Gaussian case.

Let's take a look at *Figure 13.13*, which presents the Gaussian and non-Gaussian data with fitted regression lines in causal and anti-causal directions:

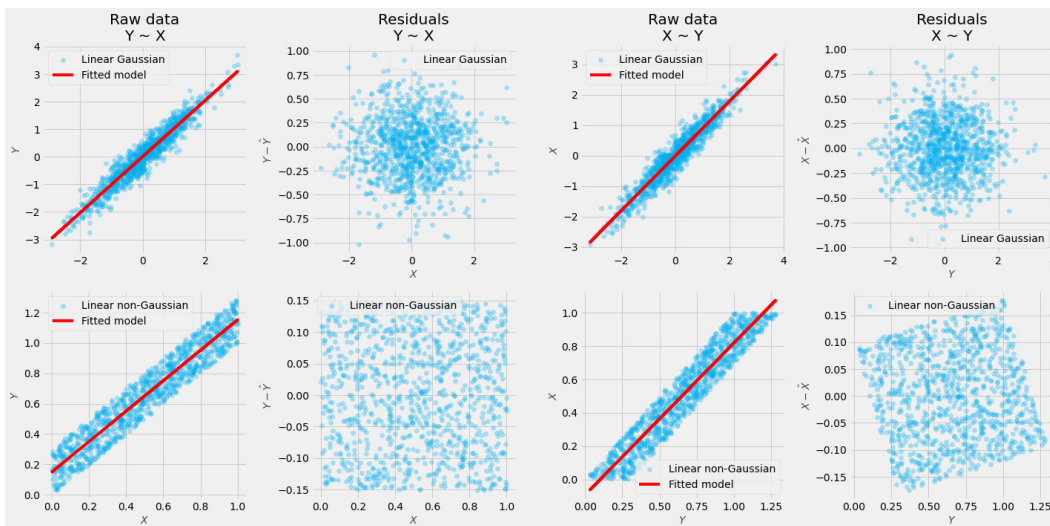


Figure 13.13 – Gaussian and non-Gaussian data with fitted regression lines

In the top panes of *Figure 13.13*, we see two Gaussian models with fitted regression lines (first and third columns) and their respective residuals (second and fourth columns). In the bottom panes, we see two non-Gaussian models (first and third columns) and their residuals (second and fourth columns).

Although both Gaussian and uniform distributions are symmetric themselves, the uniform is asymmetric rotationally. Notice how the residuals form a rotated pattern in the bottom-right corner. The intuition behind this asymmetry is closely related to the **independent component analysis (ICA)** model.

ICA is an algorithm frequently used to recover the source signals from noisy overlapping observations. A popular example of ICA usage is the source separation in the so-called **cocktail party problem**, where we have a multi-track recording of multiple speakers speaking simultaneously and we want to separate those speakers' voices into separate tracks.

It turns out that we can achieve it under certain assumptions using ICA. One of the core assumptions here is the non-Gaussianity of the source signals, which allows us to presume that there's a bijective mapping between the noisy recording and the source.

As LiNGAM relies on ICA internally, it also inherits this assumption.

The second assumption is linearity. ICA can only hold linear data and, again, LiNGAM inherits this limitation.

The good news is that LiNGAM does *not* require the faithfulness assumption.

Let's see LiNGAM in action!

LiNGAM in action

We'll start with the data that we generated for the PC algorithm to see what happens when LiNGAM's assumptions are not met.

Recall that `pc_dataset` was linear and Gaussian.

Let's instantiate and fit the model:

```
lingam = ICALiNGAM(random_state=SEED)
lingam.learn(pc_dataset)
```

Let's plot the results. *Figure 13.14* presents the comparison between the predicted and the correct adjacency matrix:

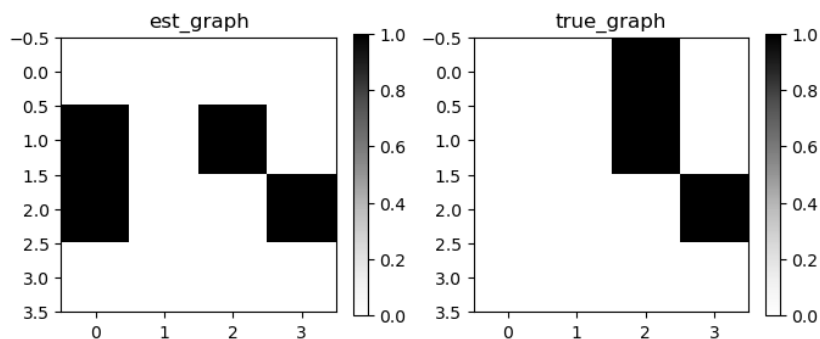


Figure 13.14 – LiNGAM results

We can see that LiNGAM has predicted two edges correctly (1 → 2 and 2 → 3) but it has missed the edge 0 → 2. Additionally, the model hallucinated two edges (1 → 0 and 2 → 0). These results are not very impressive, especially for such a simple graph. That said, this is expected as we violated one of the basic assumptions of the method.

Using legal data with LiNGAM

Let's generate the data that LiNGAM can deal with. Let's keep the same causal structure that we used for `pc_dataset`, and only update the functional forms:

```
a = np.random.uniform(0, 1, N)
b = np.random.uniform(3, 6, N)
c = a + b + .1 * np.random.uniform(-2, 0, N)
d = .7 * c + .1 * np.random.uniform(0, 1, N)
lingam_dataset = np.vstack([a, b, c, d]).T
```

Let's re-instantiate and refit the model:

```
lingam = ICALiNGAM(random_state=SEED)
lingam.learn(lingam_dataset)
```

Figure 13.15 demonstrates the model's results on the new dataset:

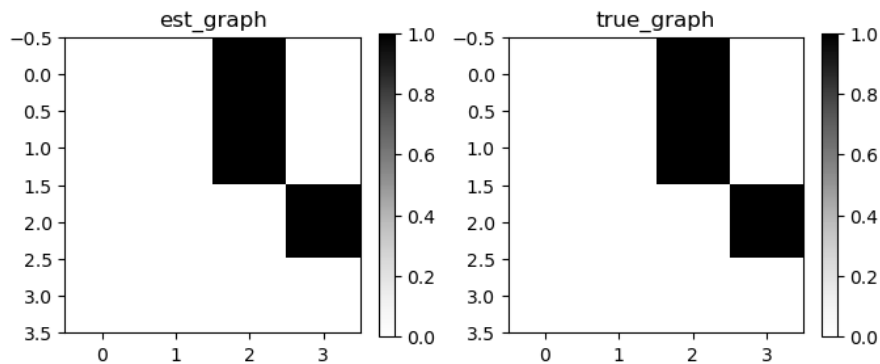


Figure 13.15 – LiNGAM on non-Gaussian data

This time, the model was able to perfectly retrieve the true graph.

A great feature that LiNGAM offers is that it does not only retrieve the causal structure but also the strength of the relationships between variables. In this sense, LiNGAM is an end-to-end method for causal discovery *and* causal inference.

Let's check how well LiNGAM did with retrieving the coefficients. We can access the learned weighted matrix via the `weight_causal_matrix` attribute:

```
lingam.weight_causal_matrix
```

This returns a weighted adjacency matrix as a `castle.common.base.Tensor` object:

```
Tensor([[0.    , 0.    , 1.003, 0.    ],
        [0.    , 0.    , 0.999, 0.    ],
        [0.    , 0.    , 0.    , 0.7   ],
        [0.    , 0.    , 0.    , 0.    ]])
```

Figure 13.16 presents these results in the context of the original structural equations. Let's see how well the algorithm managed to recover the true coefficients.

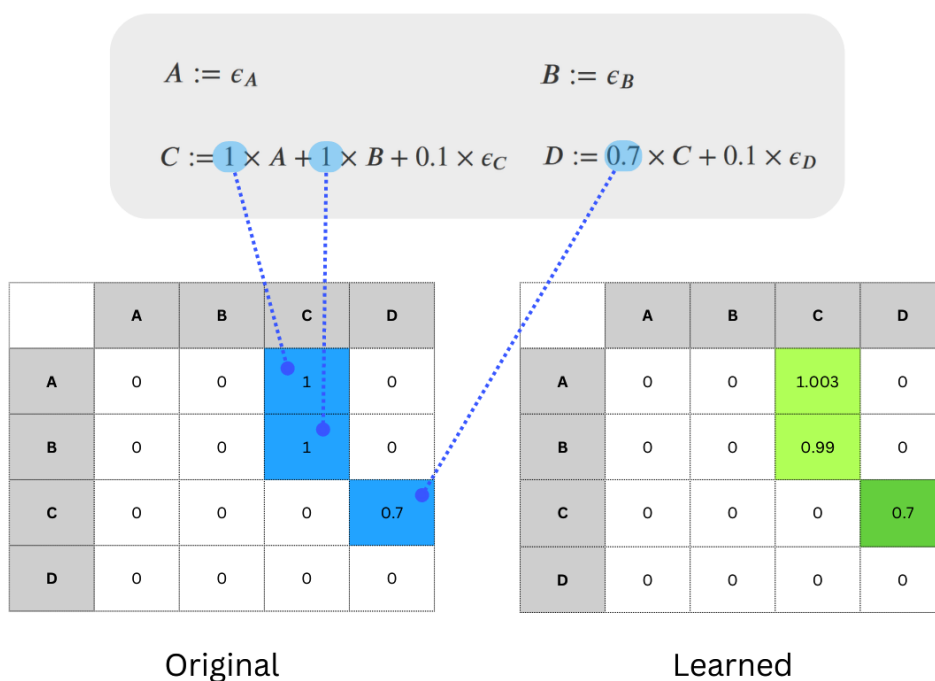


Figure 13.16 – LiNGAM results (right) versus the original DAG (left) and the original structural equations (top)

In Figure 13.16 (top), we see the set of original structural equations (the ones that we implemented). On the bottom, we see the original DAG (left) and the learned matrix (right). The blue dashed lines map the coefficients from the equations to the respective cells in the original DAG. The green cells on the right contain the coefficients retrieved by LiNGAM. Dark green emphasizes the perfectly retrieved coefficient.

These results are pretty good!

Before we conclude, there are a couple of things that are good to know about LiNGAM. First, LiNGAM uses ICA in the background, and ICA is a stochastic method. Sometimes, it might not converge to a good solution within the default number of steps.

You can modify the number of steps by modifying the `max_iter` parameter:

```
lingam = ICALiNGAM(  
    max_iter=2000,  
    random_state=SEED  
)
```

Second, LiNGAM is not **scale-invariant**, which might result in estimation errors.

Get more direct

DirectLiNGAM (Shimizu et al., 2011) is an ICA-free variation of LiNGAM. The model uses a series of regressions in order to determine the causal order.

It's guaranteed to converge to the true solution in the infinite sample regime when the model assumptions are strictly met. The number of steps required for convergence scales linearly with the number of variables.

Of course, we never have access to an infinite number of samples in the real world. That said, DirectLiNGAM has been shown to outperform its counterpart on simulated finite sample data as well (Shimizu et al., 2011).

DirectLiNGAM also solves the scaling issue. The cost for the improvements is in computational time. The time scales as a power function of the number of variables (nodes).

Using DirectLiNGAM with gCastle is straightforward:

```
d_lingam = DirectLiNGAM()  
d_lingam.learn(lingam_dataset)
```

The results are identical to the regular model, so we skip them here. You can see the plots and the learned weighted matrix in the notebook.

More variants of the LiNGAM model exist than the two we discussed here. The LiNGAM framework has been extended to work with time series, cyclical graphs (Lacerda et al., 2008), and latent variables, among others. For implementations and references for some of these variants, check the Python LiNGAM library (<https://bit.ly/LiNGAMDocs>).

It's time to conclude our journey with functional-based causal discovery for now.

In this section, we introduced functional-based causal discovery methods. We’ve seen how non-linearity and non-Gaussianity can help us break the symmetry and follow the traces left by causal mechanisms in order to gain valuable insights.

We learned how to implement the ANM model from scratch using Python and HSIC independence tests from the gCastle library and how to use the algorithms from the LiNGAM family using gCastle. Finally, we discussed the main limitations of ICA-LiNGAM and discussed how to address them using DirectLiNGAM.

In the next section, we’ll introduce the most contemporary family of causal discovery methods – gradient-based methods.

Gradient-based causal discovery

In this section, we’ll introduce **gradient-based causal discovery** methods. We’ll discuss the main contributions of this family of methods and their main disadvantages. Finally, we’ll implement selected methods using gCastle and compare their performance with other families.

What exactly is so gradient about you?

2018 was an exciting year for the causal discovery community. Xun Zheng from CMU and his colleagues presented an interesting paper during the 2018 NeurIPS conference.

The work was titled *DAGs with NO TEARS: Continuous Optimization for Structure Learning* and introduced a novel approach to causal structure learning (though we need to say that the authors did not explicitly state that their method is causal).

The proposed method (called **NOTEARS**) was not based on a set of independence tests or local heuristics but rather treated the task of structure learning as a joint, continuously-optimized task.

One of the main breakthroughs in the paper was the differentiable function encoding the acyclicity constraints. In other words, the authors proposed a function that we could optimize continuously, making sure that the graph we’re fitting to the data is acyclic.

Before we continue, it’s good to mention that gradient-based methods are essentially score-based – we compute a numerical summary of the quality of graph data fit in the optimization process.

Nevertheless, we’ve chosen to think about them as a separate family because the fact that they use continuous optimization comes with unique opportunities and challenges.

Let’s see how to encode the acyclicity constraint as a differentiable function. It turns out that the formula is very concise:

$$\mathcal{R}(A) = \text{tr}(e^{A \odot A}) - d$$

In the preceding formula, the following applies:

1. A is the adjacency matrix of a graph, G
2. e is a matrix exponential
3. $tr(\cdot)$ is a trace of a matrix
4. d is the number of nodes in a graph represented by A
5. $\odot$ is the Hadamard (element-wise) product

For a quick introduction to matrix exponentials, check <https://bit.ly/MatrixExponential>.

It turns out that when $\mathcal{R}(A)$ is equal to zero, a graph represented by the adjacency matrix A is a DAG. Otherwise, it's not. This is a very convenient way to measure what the authors refer to as the *DAG-ness* of a graph.

Let's implement this formula in Python:

```
from scipy import linalg

def check_if_dag(adj_matrix):
    A = adj_matrix
    return np.trace(linalg.expm(A * A)) - A.shape[0] == 0
```

First, we import the `linalg` linear algebra module from SciPy. We'll need it for the matrix exponential operation.

Next, we multiply the adjacency matrix element-wise by itself. Note that this step is only effective for weighted adjacency matrices. For binary matrices, $A \odot A = A$.

Next, we perform matrix exponentiation using the `linalg.expm()` function.

Finally, we compute the trace of the resulting matrix and subtract the number of rows in the adjacency matrix, which is equivalent to the number of nodes in the graph represented by this matrix (or the number of variables in our problem).

If this quantity (note that trace is a scalar value) is equal to zero, we return `True`. Otherwise, we return `False`.

Let's test the function on the `pc_dag` DAG that we created at the beginning of this chapter:

```
check_if_dag(pc_dag)
```

This returns the following:

```
True
```

The result is as expected.

Let's build a simple cyclic graph:

```
dcg = np.array([
    [0, 1, 0],
    [1, 0, 0],
    [0, 1, 0]
])
```

Let's evaluate it using our function:

```
check_if_dag(dcg)
```

This gives us the following:

```
False
```

This result is also as expected.

Gradients for $\mathcal{R}(A)$ are easy to compute and so it's naturally compatible with gradient-based optimization methods.

The overall loss function in NOTEARS consists of the mean squared error component $\|X - XA\|_F^2$ with a number of additional regularization and penalty terms. The function is optimized using the augmented Lagrangian method (Nemirovski, 1999) subject to the constraint that $\mathcal{R}(A) = 0$ (see Zheng et al., 2018, for the full definition).

In the main loss component, X is an $n \times d$ data matrix, where n is the number of observations and d is the number of features, A is the adjacency matrix, and $\|\cdot\|_F^2$ is the squared Frobenius norm (<https://bit.ly/MatrixNorm>; see also *Chapter 14* for definition).

Shed no tears

The authors have tested NOTEARS against a selection of traditional methods including PC, LiNGAM, and **FGES (fast GES)**. In their experiments, the authors found that NOTEARS either outperformed or performed on par with the best traditional methods (depending on the setting).

These initial results brought a lot of excitement to the community and inspired new lines of research.

The initial excitement cooled down after the publication of two research papers that examined the NOTEARS algorithm. Kaiser and Sipos (2021) have shown that NOTEARS' sensitivity to scaling makes it a risky choice for real-world causal discovery. Reisach et al. (2021) demonstrated similar inconsistencies in the performance of gradient-based methods between standardized and unstandardized data and pointed to a more general problem: synthetic data used to evaluate these methods might contain unintended regularities that can be relatively easily exploited by these models, as the authors expressed in the title of their paper, *Causal Discovery Benchmarks May Be Easy To Game*.

GOLEMs don't cry

GOLEM (Ng et al., 2020) is an algorithm that improves over NOTEARS. The algorithm uses a likelihood-based objective (rather than an MSE-based one like in NOTEARS) and soft DAG-ness constraint (rather than a hard one like in NOTEARS), which leads to facilitated optimization and faster convergence. The authors demonstrate that GOLEM outperforms NOTEARS on synthetic data.

These changes do not seem to address all the issues raised by Reisach et al. (2021) and it seems that GOLEM's performance is affected by the unintended variance patterns introduced by synthetic data-generating processes. In particular, the model's performance tends to degrade in the normalized data scenario, where *unfair* variance patterns are minimized. Note that the same is true for NOTEARS (check Reisach et al., 2021, for the details).

NOTEARS, GOLEM, and linearity

The original version of NOTEARS was designed to work with linear data. It's the same for GOLEM. An extension of NOTEARS exists that works with general non-parametric models (Zheng et al., 2020).

The comparison

Now, let's generate three datasets with different characteristics and compare the performance of the methods representing each of the families.

We'll start by building the dataset:

```
true_dag = DAG.scale_free(n_nodes=11, n_edges=15, seed=SEED)

DATA_PARAMS = {
    'linearity': ['linear', 'nonlinear'],
    'distribution': {
        'linear': ['gauss', 'exp'],
        'nonlinear': ['quadratic']
    }
}

datasets = {}

for linearity in DATA_PARAMS['linearity']:
    for distr in DATA_PARAMS['distribution'][linearity]:
        datasets[f'{linearity}_{distr}'] = IIDSimulation(
            W=true_dag,
            n=2000,
            method=linearity,
            sem_type=distr)
```

First, we generate a graphical structure according to the scale-free model. Our graph will have 10 nodes and 15 edges. *Figure 13.17* presents the graph:

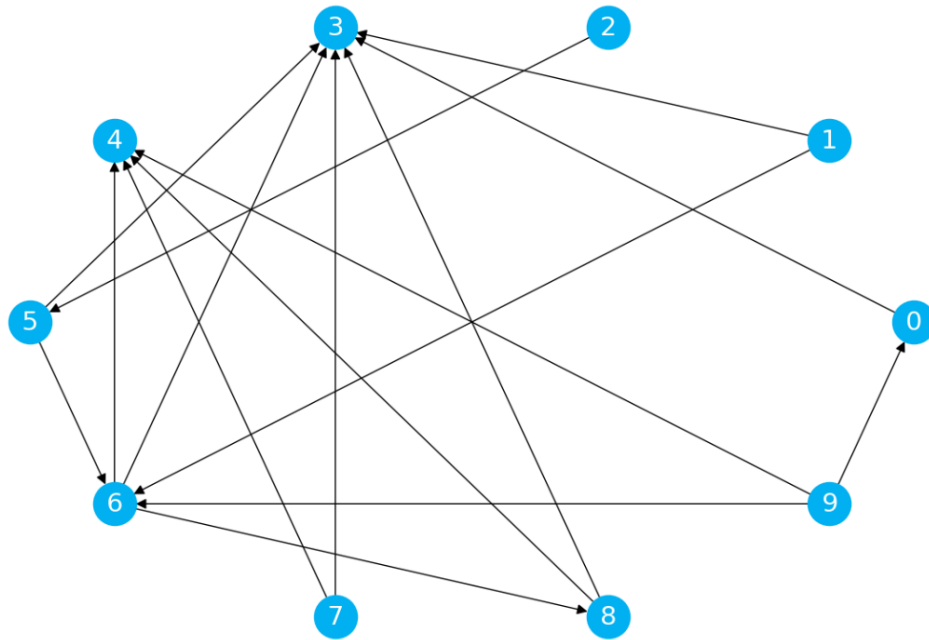


Figure 13.17 – The graph for the comparative experiment

Next, we define what type of functional relationships and/or noise we want to have in our SCM (linear with Gaussian and exponential noises and non-linear quadratic).

Finally, we iterate over the defined settings and generate the observations. We store the observations in the `datasets` dictionary.

Now, let's create an object containing the methods we'll use:

```
methods = OrderedDict({
    'PC': PC,
    'GES': GES,
    'LiNGAM': DirectLiNGAM,
    'Notears': NotearsNonlinear,
    'GOLEM': GOLEM
})
```

We're now ready to start the experiment. You can find the full experimental loop in the notebook for this chapter.

We present the results in *Figure 13.18*. The best-performing models for each dataset are marked in bold.

Dataset	Model	SHD	FDR	F1	No. undir. edges
Linear Gauss.	PC	5	0.2	0.75	1
Linear Gauss.	GES	17	0.5833	0.4762	2
Linear Gauss.	DirectLiNGAM	11	0.3889	0.6471	0
Linear Gauss.	NotearsNonlinear	11	0.4667	0.5161	0
Linear Gauss.	GOLEM	0	0	1	0
Lineaer Exp.	PC	5	0.1875	0.7647	2
Lineaer Exp.	GES	18	0.625	0.4286	2
Lineaer Exp.	DirectLiNGAM	0	0	1	0
Lineaer Exp.	NotearsNonlinear	12	0.44	0.5882	0
Lineaer Exp.	GOLEM	0	0	1	0
Non-lin. Quad	PC	14	0.5714	0.3871	1
Non-lin. Quad	GES	13	0.5714	0.3871	1
Non-lin. Quad	DirectLiNGAM	16	1	NaN	0
Non-lin. Quad	NotearsNonlinear	7	0.1818	0.6667	0
Non-lin. Quad	GOLEM	18	0.6111	0.4118	0

Figure 13.18 – Results of the model comparison

In this section, we introduced gradient-based methods for causal discovery. We discussed the continuous acyclicity constraint (DAG-ness), and its innovative character. We discussed the main challenges that gradient-based methods face. Finally, we carried out an experiment comparing algorithms from different families on a synthetic dataset.

In the next section, we'll see how to combine causal discovery algorithms with expert knowledge.

Encoding expert knowledge

Combining expert knowledge with automated methods can be incredibly beneficial. It can help algorithms learn while inspiring human stakeholders to deepen their own insights and understanding of their environments and processes.

In this section, we'll demonstrate how to incorporate expert knowledge into the workflow of our causal discovery algorithms.

By the end of this section, you'll be able to translate expert knowledge into the language of graphs and pass it to causal discovery algorithms.

What is expert knowledge?

In this section, we think about *expert knowledge* as an umbrella term for any type of knowledge or insight that we're willing to accept as valid.

From the algorithmic point of view, we can think of expert knowledge as a strong (but usually local) prior. We encode expert knowledge by *freezing* one or more edges in the graph. The model treats these edges as existing and adapts their behavior accordingly.

Expert knowledge in gCastle

Let's see how to encode external knowledge using gCastle.

Currently (gCastle 1.0.3), one algorithm supports adding external knowledge – the PC algorithm – but support for more algorithms is planned.

We'll take the linear Gaussian dataset from the previous section's experiment and we'll try to improve PC's performance by adding some external knowledge to the algorithm.

We start with an additional import. The `PrioriKnowledge` object will allow us to conveniently define and pass the knowledge to the algorithm:

```
from castle.common.priori_knowledge import PrioriKnowledge
```

Next, we instantiate the `PrioriKnowledge` object and pass the number of nodes to the constructor:

```
priori_knowledge = PrioriKnowledge(n_nodes=10)
```

Next, we add the required and forbidden edges:

```
priori_knowledge.add_required_edges([(7, 3)])  
priori_knowledge.add_forbidden_edges([(0, 9), (8, 6)])
```


I checked the plots for the experiment in the notebook to find the edges the PC algorithm got wrong in the previous section's experiment. We added some of them here.

Now, we're ready to instantiate the model and train it:

```
pc_priori = PC(priori_knowledge=priori_knowledge)
pc_priori.learn(datasets['linear_gauss'].X)
```

Note that, this time, we pass the `priori_knowledge` object to the constructor.

Let's compare the results before and after sharing our knowledge with the algorithm. *Figure 13.19* summarizes the results for us:

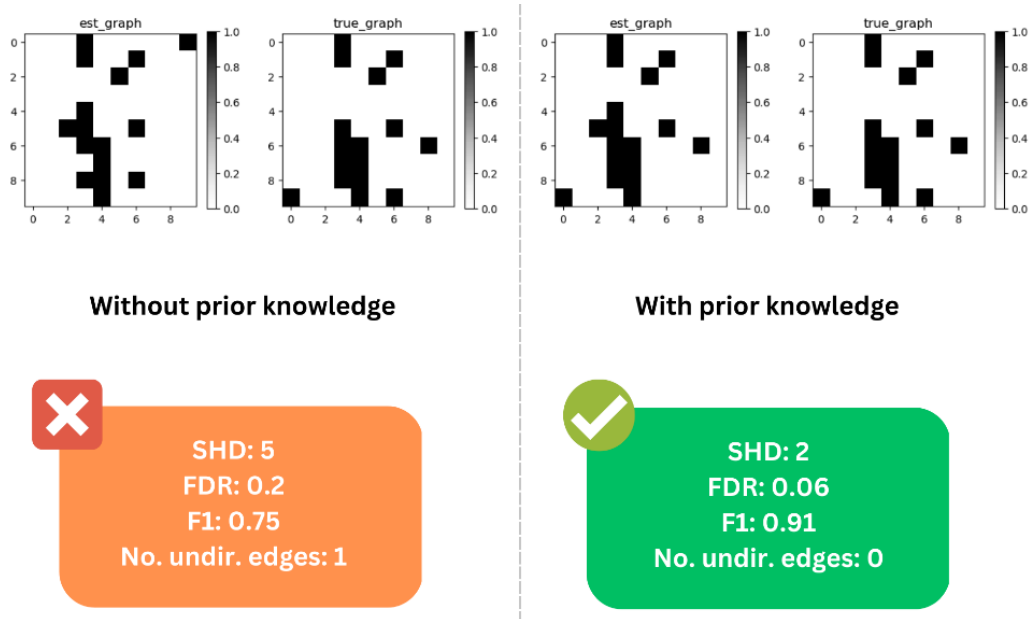


Figure 13.19 – Results for the PC algorithm before and after adding prior knowledge

In *Figure 13.19*, we see that all metrics that we record have been improved after adding external knowledge to the algorithm. We could continue adding and restricting more edges to improve the algorithm further.

In this section, we've learned how to encode and pass expert knowledge to the PC algorithm. Currently, only one algorithm in gCastle supports external knowledge, but as far as I know, the developers are planning to add support for more algorithms in the future.

Wrapping it up

We started this chapter with a refresher on important causal discovery assumptions. We then introduced gCastle. We discussed the library’s main modules and trained our first causal discovery algorithm. Next, we discussed the four main families of causal discovery models – constraint-based, score-based, functional, and gradient-based – and implemented at least one model per family using gCastle. Finally, we ran a comparative experiment and learned how to pass expert knowledge to causal models.

In the next chapter, we’ll discuss more advanced ideas in causal discovery and take a broader perspective on the applicability of causal discovery methods in real-life use cases.

Ready for one more dive?

References

- Barabási, A. L. (2009). *Scale-free networks: a decade and beyond*. Science, 325(5939), 412-413.
- Barabási, A. L., and Albert, R. (1999). *Emergence of Scaling in Random Networks*. Science, 286(5439), 509-512.
- Blake, W. (2009) *The Tyger*. Songs of Experience. In William Blake: The Complete Illuminated Books.
- Cai, R., Wu, S., Qiao, J., Hao, Z., Zhang, K., and Zhang, X. (2021). *THP: Topological Hawkes Processes for Learning Granger Causality on Event Sequences*. ArXiv.
- Chickering, D. M. (2003). *Optimal structure identification with greedy search*. J. Mach. Learn. Res., 3, 507-554.
- Chickering, M. (2020). *Statistically Efficient Greedy Equivalence Search*. Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence (UAI). In Proceedings of Machine Learning Research, 124, 241-249.
- Colombo, D., and Maathuis, M.H. (2012). *Order-independent constraint-based causal structure learning*. J. Mach. Learn. Res., 15, 3741-3782.
- Cressie, N., and Read, T.R. (1984). *Multinomial goodness-of-fit tests*. Journal of the Royal Statistical Society: Series B (Methodological), 46(3), 440-464.
- Enquist, M., Arak, A. (1994). *Symmetry, beauty and evolution*. Nature, 372, 169-172
- Erdős, P. and Rényi, A. (1959). *On Random Graphs I*. Publicationes Mathematicae Debrecen, 6, 290-297.
- Fisher, R. A. (1921). *On The “Probable Error” of a Coefficient of Correlation Deduced From a Small Sample*. Metron, 1, 1-32.